

WHMCS Performance Telemetry Analysis Report (K8s Service: TSA-WHMCS)

This report details the latest Elastic APM performance analysis of the `TSA-WHMCS` service running on Kubernetes (K8s) using standard analysis thresholds:

- **Transactions & APIs:** $\geq 2000\text{ms}$
- **Database Queries:** $\geq 100\text{ms}$

1. Latency Baseline Summary

- **Median Response Time (p50):** 467.29 ms
- **95th Percentile Latency (p95):** 3,640.89 ms (3.6s)
- **99th Percentile Latency (p99):** 11,068.87 ms (11.0s)

2. Isolated Database Bottlenecks ($\geq 100\text{ms}$)

The latest telemetry isolates several high-impact database queries causing thread-blocking and user delays:

A. Severe Table Locks on Activity Log Deletions

- **Query:** `delete from tblactivitylog where userid = ? and id <= ?`
- **Average Duration:** **6,244.49 ms** (6.2s)
- **Root Cause:** While database inspection shows that an index on `userid` **already exists**, the table has 6 distinct indexes. A delete command on a large number of rows requires updating all 6 indexes for every deleted row, which blocks database execution threads. This is compounded by severe concurrent write locks from continuous system logging.
- **Remediation:** Run database pruning of historical logs weekly to control table size, and defragment using `OPTIMIZE TABLE` during off-peak hours to speed up index modifications.

B. Slow DNSManager3 Job Lookups

- **Queries:**
 - `select * from DNSManager3_Job where job = ? and status = ?` (Average: **843.64 ms**)
 - `select DNSManager3_Job.* from DNSManager3_Job where userid = ?` (Average: **518.57 ms**)
- **Root Cause:** Missing indexes on the job execution table used by the custom DNS manager module.
- **Remediation:** Create a composite index on `(job, status)` and a single index on `userid`.

C. Client Admin Directory Sorting

- **Query:** `SELECT SQL_CALC_FOUND_ROWS ... FROM tblclients ORDER BY lastname ASC, firstname ASC, companyname ASC LIMIT ...`
- **Average Duration:** **138ms - 147ms** (high frequency)
- **Root Cause:** The admin client listing page queries a large number of rows and performs a filesystem sort (filesort) to order the records by name.
- **Remediation:** Create a composite index on `(lastname, firstname, companyname)` to enable direct index ordering and avoid filesystem sorts.

D. Domain Renewal Reminder History Lookups

- **Query:** `SELECT * FROM tblDomainReminders WHERE domain_id = ? ORDER BY id DESC`
 - **Average Duration:** 103.95 ms
 - **Root Cause:** Missing index on `domain_id` column.
 - **Remediation:** Create an index on `domain_id` .
-

3. Outbound API & Transaction Bottlenecks (>= 2000ms)

Synchronous, blocking external network connections remain the primary contributor to the application's p99 latency spikes:

A. Olitt API Integration delays (22.3s - 28.4s)

- **Outbound HTTP Spans:**
 - `POST olitt.com` (Average: 28,454.69 ms)
 - `POST app.olitt.com` (Average: 22,308.44 ms)
- **Transactions Blocked:**
 - `POST /cloud/modules/addons/olitt_whmcs_addon/olitt-connect.php` (33.7s average latency)
 - `GET /cloud/modules/addons/olitt_ai_sites/olitt-edit.php` (22.5s average latency)
- **Impact:** Transactions block PHP execution indefinitely waiting for remote responses.
- **Remediation:** Enforce connection and execution timeouts (3s / 5s) on all outbound cURL clients (as implemented in `olitt-connect.php`).

B. Lim116 Host Node Connection (14.2s Overhead)

- **Outbound HTTP Span:** `POST 141.95.3.80` (hostname: `lim116.truehost.cloud`)
- **Average Duration:** 14,257.64 ms (14.2s)
- **Impact:** Order placements and client operations are blocked by slow hypervisor communication.

C. DNS Manager Sync (4.9s Overhead)

- **Outbound HTTP Span:** `GET central3.dns.truehost.cloud`
- **Average Duration:** 4,900.88 ms (4.9s, executed 196 times)
- **Root Cause:** Synchronous DNS syncing operations to your central nameserver during client area requests.

D. Hypervisor Status Checks (2.9s Overhead)

- **Outbound HTTP Span:** `POST 157.180.74.207`
 - **Average Duration:** 2,929.05 ms (2.9s, executed 148 times)
 - **Remediation:** Mitigated by the 60-second caching wrapper implemented on the `Virtualizor_Curl` class.
-

4. Kubernetes-Specific Recommendations & Remediation Plan

Step 1: Stage, Commit, and Push Code Changes

Push the optimization branch to your repository:

```
git add entrypoint.sh modules/addons/olitt_whmcs_addon/olitt-connect.php
modules/servers/virtualizor/virtualizor.php php.ini
```

```
git commit -m "feat: optimize WHMCS performance, add caching, and database configuration"
git push origin optimize-whmcs
```

Step 2: Build and Deploy the Container Image

Trigger your CI/CD pipeline to compile the new Docker image including the updated `php.ini` , `entrypoint.sh` , and `cache-enabled` files.

Step 3: Configure K8s Deployment Manifest & Environment Variables

Update your Kubernetes deployment/values manifest to pass the caching and telemetry environment variables:

```
spec:
  template:
    spec:
      containers:
        - name: whmcs-app
          image: truehostcloud/whmcs:your-new-tag
          env:
            - name: INSTRUMENT_ELASTIC_APM
              value: "True"
            - name: APM_SERVICE_NAME
              value: "TSA-WHMCS"
            - name: APM_SERVER_URL
              value: "http://localhost:8200" # Local loopback to sidecar
            - name: REDIS_HOST
              value: "redis-service.default.svc.cluster.local" # Internal K8s

service
```

Step 4: Deploy APM Server Sidecar Container

To eliminate telemetry collection network delay, run the APM server as a sidecar container in the same pod. Add this to the `spec.containers` block of your deployment:

```
- name: apm-server
  image: docker.elastic.co/apm/apm-server:8.11.0
  ports:
    - containerPort: 8200
  env:
    - name: elastic_apm.server_url
      value: "https://apm.jisort.com"
    - name: apm-server.auth.secret_token
      value: "YOUR_APM_SECRET_TOKEN"
```

Step 5: Execute Database Optimization & Maintenance

Run the optimized DDL index scripts and trigger defragmentation/statistics update.

1. **Deploy Missing Indexes:** Execute the indexing statements defined in the database optimization guide [whmcs_db_optimization.md](#) (or run the sql commands directly from your database client).
2. **Defragment & Analyze Tables** (Run during off-peak hours):

```
kubectl exec -i deployment/whmcs-deployment -- mysql -h "$DB_HOST" -u  
"$DB_USER" -p"$DB_PASSWORD" "$DB_NAME" -e "ANALYZE TABLE tblinvoiceitems,  
tblactivitylog, DNSManager3_Job, tblclients, tbldomainreminders; OPTIMIZE  
TABLE tblinvoiceitems, tblactivitylog, DNSManager3_Job, tblclients,  
tbldomainreminders;"
```